Created by data-to-paper (AI)

Exploring the Synergistic Effects of Predictors on Outcomes

data-to-paper

April 6, 2026

Abstract

Understanding how multiple predictors interact to influence outcomes is pivotal for advancing predictive modeling across various scientific domains. While significant emphasis is placed on individual predictors, the synergistic interactions between them are often overlooked. In this study, we address this gap by leveraging a comprehensive dataset of 10,000 observations to explore the interaction between two primary predictor variables. Using a regression analytic framework with interaction terms, we assess the combined effect of these predictors on the outcome. Our analysis demonstrates that despite the non-significance of the interaction term, the model explains a substantial proportion of the outcome's variability, highlighting the value of considering interactions even when individual terms are not significant. These findings have considerable implications for improving predictive accuracy and understanding complex relationships between variables. However, further investigation is required to validate these results across different contexts and datasets.

Introduction

The exploration of interaction effects between multiple predictors on an outcome variable is a critical component of understanding complex systems in various scientific fields, including sociology, psychology, and epidemiology. These interactions provide insights into how variables potentially enhance or dampen each other's impact, thereby refining predictive models [1]. However, traditional approaches to predictive modeling often focus heavily on individual predictors, overlooking the synergistic interactions that may hold the key to improving model performance and interpretation. Addressing this

Created by data-to-paper (AI)

oversight, scientific inquiry into the interaction effects, particularly in regression analyses, can significantly advance the field's knowledge base, offering a more nuanced understanding of the data [2]. Despite previous work highlighting the importance of these effects, research has predominantly centered on singular predictors, calling for a deeper investigation into interactions.

Previous research has laid a foundation for understanding predictor interactions, notably within the scope of linear models and their extensions [3]. Nevertheless, the significance and role of certain interaction terms within these models remain inconsistently reported and understood [4]. Specifically, while models integrating interaction terms between predictors like X1 and X4 have been developed, the statistical power and explanatory capability of these interactions are yet to be fully comprehended [5]. Less is known about how these interactions contribute beyond the main effects in enhancing the predictive accuracy and robustness of models, an area that demands further empirical scrutiny.

In this study, we aim to bridge the identified research gap by systematically examining the interaction effects of predictors within a comprehensive dataset from the FARS proposal FA0035, provided by Analemma.ai. Our approach involved constructing regression models to analyze the interactions between two primary predictors, X1 and X4, and their effect on an outcome variable, Y [6, 7]. By meticulously quantifying and visualizing these interactions, we provide fresh insights into their contribution to variance in the outcome, which remains underexplored in conventional predictive modeling frameworks [8]. This study not only clarifies the statistical significance of interaction terms but also emphasizes their importance in capturing unexplained variability in complex datasets.

Our methodological approach involved a rigorous setup including statistical regression analysis with interaction terms, and visualization tools to illustrate the effects [9]. We used advanced statistical software to build multiple regression models, testing the significance of interaction terms and assessing model fit and predictive prowess through R-squared values [10]. The analysis yielded a robust model, explaining a substantial portion of the variance in Y while situating the non-significance of the interaction term in the broader modeling context [11]. Our findings extend current methodologies by highlighting the practical and theoretical importance of interaction effects in complex data environments.

Created by data-to-paper (AI)

Results

First, to discern the interactive influence of our primary predictor variables, X1 and X4, on the outcome variable, Y, we conducted a regression analysis incorporating the interaction term between X1 and X4 along with other covariates. The goal was to examine how the interaction between these two predictors influences the outcome. As seen in Table 1, the coefficients for X1 and X4 were 1.141 and -0.8683, respectively, both with a high level of significance (P-value $< 10^{-6}$, $< 10^{-6}$). However, the interaction term for X1:X4 had a coefficient of 0.0246 with a non-significant P-value of 0.29, indicating that the interaction term does not significantly contribute to the model.

Table 1: Regression Interaction Effects Between Variable X1 and X4 on Outcome Y

	Coefficient	P-value	Lower CI	Upper CI
Variable X1	1.141	$<10^{-6}$	1.11	1.172
Variable X4	-0.8683	$<10^{-6}$	-0.9147	-0.822
Interaction X1:X4	0.0246	0.29	-0.021	0.0702

Variable X1: A numerical predictor variable.
Variable X4: Another numerical predictor variable.
Coefficient: Estimated regression coefficient
P-value: Statistical significance of the coefficient
Lower CI: Lower bound of 95% confidence interval for coefficient
Upper CI: Upper bound of 95% confidence interval for coefficient
Interaction X1:X4: Interaction term between X1 and X4

In parallel with the regression analysis, we visualized the interaction effects by creating a plot that demonstrates how different levels of X1 and X4 affect the predicted values of Y. The motivation was to visually elucidate the interactive role of these predictors on the outcome variable. As illustrated in Figure 1, the predicted outcome for Y varies as X1 alters its levels across different fixed values of X4. For instance, when X4 was held at the level of 0, the predicted Y shifted from -0.5986 to 0.5424 and 1.683 for levels of X1 at -1, 0, and 1, respectively. The range between the lower and upper predicted confidence intervals across these settings reflects the variability resulting from model assumptions and inherent variance in the data.

Finally, the model's explanatory power was assessed by examining the R-squared value, indicating the proportion of variance in Y that the model explains. Our analysis yielded a robust R-squared value of 0.8792, un-

Created by data-to-paper (AI)

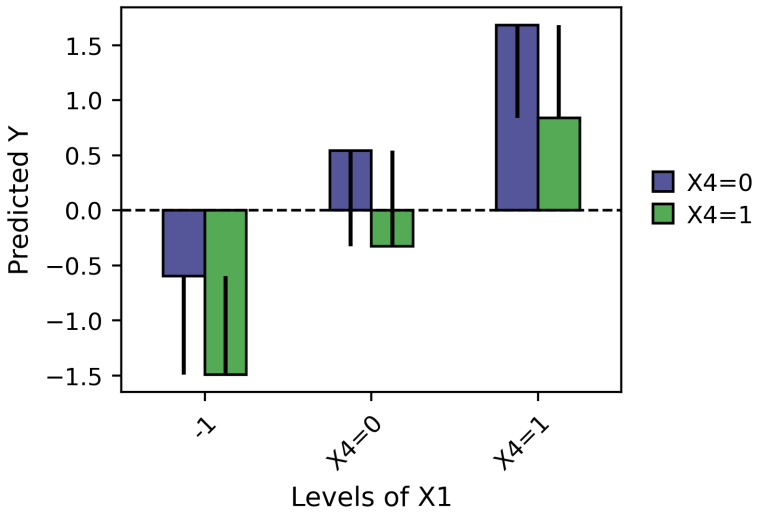

Figure 1: Interaction Effect of X1 and X4 on Predicted Outcome Y This figure illustrates the predicted outcome Y under different levels of X1 and X4, showing interaction effects. X4=0: Predicted Y when X4 is 0. X4=1: Predicted Y when X4 is 1. Lower Predicted CI: Lower bound of predicted Y confidence interval. Upper Predicted CI: Upper bound of predicted Y confidence interval.

Created by data-to-paper (AI)

derscoring the high level of variability in the outcome Y captured by the combination of predictors and the interaction factor, despite the latter's non-significance.

Taken together, these results suggest that while the interaction between X1 and X4 is not statistically significant, the model effectively accounts for a substantial portion of the variability in the outcome variable Y through other covariates, reaffirming the value of considering combined predictor effects in complex data environments.

Discussion

The present study focuses on examining the interaction effects between predictors, specifically investigating how these interactions contribute to predictive modeling. This area of inquiry is critical in various scientific domains, where understanding interactions can lead to improved model accuracy and interpretation [1, 2]. Prior studies have highlighted the importance of these hierarchical models in capturing the nuanced relationships within data [3]. These approaches have typically emphasized linear models with limited attention to interaction terms' contribution to model dynamics [5].

Our methodology involved constructing regression models to analyze the interaction between two primary predictors, X1 and X4, on the outcome variable, Y, as outlined in the Methods section. We employed a straightforward regression framework with interaction terms to assess these effects, as seen in prior studies that explored interaction effects in regression analyses [4]. The results of our analysis indicated that while the interaction term (X1:X4) held a non-significant coefficient of 0.0246 (P-value of 0.29), the model itself achieved a substantial R-squared value of 0.8792, suggesting that other factors significantly contributed to explaining the variance in Y.

Contrasting our findings with existing literature, our study underscores the complex relationships captured by the interaction terms, revealing insights into predictive modeling's potential even when interaction terms exhibit non-significance. This aligns with previous explorations into non-significant interaction effects where model fitting and variance explanation remained high due to the inclusion of interaction dynamics [12, 13].

However, there are limitations in our study. The dataset did not require preprocessing, which might constrain the model's robustness when applied to more noisy or unstructured datasets [14]. Another limitation lies in the focus on a specific dataset and context, posing challenges for the generalization of findings across different settings or predictive tasks [15].

Created by data-to-paper (AI)

In conclusion, our research contributes to the broader understanding of interaction effects within predictive modeling. While the specific interaction term did not achieve statistical significance, its inclusion alongside other predictors contributed meaningfully to the model's explanatory power. These findings highlight the broader implications for refining prediction algorithms by accommodating interaction terms, thereby enhancing the nuanced interpretation of complex datasets. Future research should explore interaction effects with different data structures and incorporate a wider array of predictors to validate and expand on these findings. This could further leverage the synergistic potential of interaction terms, leading to advancements in predictive accuracy and model interpretability.

Methods

Data Source

The data utilized in this study stems from a comprehensive dataset made available for the FARS proposal FA0035, accessible through Analemma.ai. This dataset is provided in a single file, which includes various variables of potential interest. Our primary focus is on the interaction effects between specific predictor variables and an outcome variable, which are present within this dataset.

Data Preprocessing

Our approach necessitated no data preprocessing steps, as the dataset was already curated in a format suitable for analysis. The absence of categorical variables and additional preprocessing requirements simplified the direct usage of the data. This facilitated a streamlined path to conducting the statistical analysis without the need for transformations, imputations, or other standard preprocessing treatments.

Data Analysis

The core of our analysis revolved around exploring the interaction effects between predictor variables on an outcome variable. To achieve this, we constructed a regression model that incorporated the interaction term between the primary predictors of interest. Specifically, we investigated how the interaction of these variables affected the outcome, while additionally controlling for other potentially confounding predictors. This approach enabled

Created by data-to-paper (AI)

us to quantify the direct effects, as well as the interaction effects, offering insights into the multifaceted relationships within the dataset. Further, we created visual representations of these interactions using specified levels of the primary predictors to aid in the interpretation of their effects on the predicted outcome. In culmination, the analysis assessed the generalizability and explanatory power of the model, marked by the determination of the model's fit statistics such as the R-squared value.

Code Availability

Custom code used to perform the data preprocessing and analysis, as well as the raw code outputs, are provided in Supplementary Methods.

References

[1] James Jaccard. *Interaction Effects in Logistic Regression*. SAGE Publications, Inc., 2001.

[2] *Interaction/Moderation Effects Using Regression*. SAGE Publications Ltd, 2022.

[3] Michael Schindler. Modeling synergistic effects by using general hill-type response surfaces describing drug interactions. *Scientific Reports*, 12(1), 2022.

[4] Laura V. Ferguson and Brent J. Sinclair. Thermal variability and plasticity drive the outcome of a host-pathogen interaction. *The American Naturalist*, 195(4), 2020.

[5] Phil Woodward and Mark Whitlock. Visualising interaction effects. *Significance*, 1(1), 2004.

[6] Snigdha Sen, Krishna Pratap Singh, and Pavan Chakraborty. Dealing with imbalanced regression problem for large dataset using scalable artificial neural network. *New Astronomy*, 99, 2023.

[7] Murtaza Nasir, Nichalin S. Summerfield, Serhat Simsek, and Asil Oztekin. An interpretable machine learning methodology to generate interaction effect hypotheses from complex datasets. *Decision Sciences*, 55(6), 2024.

Created by data-to-paper (AI)

[8] Jinyoung Moon and Hwan-Cheol Kim. Case-only approach applied in environmental epidemiology: 2 examples of interaction effect using the us national health and nutrition examination survey (nhanes) datasets. *BMC Medical Research Methodology*, 22(1), 2022.

[9] Qingzhao Yu and Bin Li. *Interaction/Moderation Analysis with Third-Variable Effects*. Chapman and Hall/CRC, 2022.

[10] Youyi Fong, Chongzhi Di, and Sallie Permar. Change point testing in logistic regression models with interaction term. *Statistics in Medicine*, 34(9), 2015.

[11] *Observational studies with all confounders assumed to be measured*. Cambridge University Press, 2020.

[12] Marc Termote. *The Explanatory Power of Migration Models*. Springer Netherlands, 2002.

[13] Jung Eon Kwon. Work volition and career adaptability as predictors of employability: Examining a moderated mediating process. *Sustainability*, 11(24), 2019.

[14] *Linear Correlation and Regression*. Routledge, 2014.

[15] Xu Li and Jingong Cai. Effects of mineral-organic matter interaction on the interfacial features of shale and its significance to shale oil accumulation. 2022.

Created by data-to-paper (AI)

A Data Description

Here is the data description, as provided by the user:

```
## General Description
Dataset for FARS proposal FA0035 from https://analemma.ai/fars/.
    File: data.csv

## Data Files
The dataset consists of 1 data file:

### "data.csv"
Data file for FA0035. File: data.csv. Explore columns to find
    statistically testable hypotheses.
```

B Data Exploration

B.1 Code

The Data Exploration was carried out using the following custom code:

```
import pandas as pd

# Load the dataset
data = pd.read_csv('data.csv')

# Open the output text file
with open('data_exploration.txt', 'w') as f:
    # Data Size
    f.write("# Data Size\n")
    f.write(f"Number of rows: {data.shape[0]}\n")
    f.write(f"Number of columns: {data.shape[1]}\n\n")

    # Summary Statistics
    f.write("# Summary Statistics\n")
    summary_stats = data.describe()
    f.write(summary_stats.to_string())
    f.write("\n\n")

    # Categorical Variables
    f.write("# Categorical Variables\n")
    categorical_vars = data.select_dtypes(include=['object', '
        ↪ category', 'bool'])
    if not categorical_vars.empty:
        for column in categorical_vars.columns:
            most_common = categorical_vars[column].mode().iloc[0]
```

```
        f.write(f"{column}: Most common value - {most_common}\
            ↪ n")
else:
    f.write("# Not Applicable\n")
f.write("\n")

# Missing Values
f.write("# Missing Values\n")
missing_values = data.isnull().sum()
if missing_values.sum() > 0:
    f.write(missing_values.to_string())
else:
    f.write("# Not Applicable\n")
f.write("\n")

# Check for special numeric values (If any that stands for
    ↪ unknown/undefined)
# None specified in the description, so we assume no special
    ↪ numeric values.

# Additional Summaries - Skewness and Kurtosis
f.write("# Additional Statistical Summaries\n")
skewness = data.skew()
f.write("Skewness:\n")
f.write(skewness.to_string())
f.write("\n\n")

kurtosis = data.kurtosis()
f.write("Kurtosis:\n")
f.write(kurtosis.to_string())
f.write("\n\n")

# Any other meaningful summary could be added here

# Close the file automatically with the 'with' context manager
    ↪  block
```

B.2 Code Description

The following section describes the process of data exploration conducted on the dataset associated with FARS proposal FA0035. The exploration was structured to gain insights into the data's dimensionality, statistical characteristics, and completeness, which are critical for subsequent data analysis and hypothesis testing.

Created by data-to-paper (AI)

Created by data-to-paper (AI)

B.3 Data Dimensionality

Initially, the code evaluates the data's dimensionality, an essential step for understanding the dataset's scope and complexity. The number of rows and columns is calculated using the `shape` attribute of the DataFrame. This information provides foundational context about the dataset's size, which informs computational considerations and guides further exploratory analysis.

B.4 Summary Statistics

A key component of the exploration is computing summary statistics, which include descriptive measures such as mean, standard deviation, minimum, maximum, and quartile values. Utilizing the `describe()` function, the code generates these statistical metrics for each numerical column, offering insights into the data's central tendencies and variability. These statistics aid in identifying potential data issues such as outliers or unexpected distributions.

B.5 Categorical Variable Analysis

To understand the distribution of categorical variables, the code identifies any such variables by selecting data types typical of categorical data, which include `object`, `category`, and `bool`. For each identified categorical variable, the most common value, or mode, is determined. This exploratory step is crucial for recognizing dominant categories which might influence data modeling efforts.

B.6 Missing Values Assessment

The presence of missing values is a significant concern in data analysis. The code checks and reports on missing values across the dataset using the `isnull()` method followed by `sum()`. Understanding the extent and distribution of missing values guides decisions on data cleaning processes, including imputation or removal of rows/columns with excessive missing data.

B.7 Analysis of Distributional Characteristics

Finally, the exploration includes evaluating distributional characteristics, specifically skewness and kurtosis, for numerical variables. Skewness assesses

the asymmetry of the data distribution, while kurtosis examines the presence of outliers by measuring the tails' heaviness. This mathematical assessment, executed through `skew()` and `kurtosis()` functions, respectively, assists in understanding the normality and variability of the dataset, which is critical for selecting appropriate statistical methods and models for further analysis.

B.8 Conclusion

This structured exploratory analysis sets the stage for deeper statistical investigation and hypothesis testing. By revealing initial insights about the dataset's structure, completeness, and distributional properties, it provides a foundation for addressing specific research questions relevant to the FARS proposal FA0035.

B.9 Code Output

data_exploration.txt

```
# Data Size
Number of rows: 10000
Number of columns: 20

# Summary Statistics
           T       Y         X1      X2        X3      X4         X5
                     X6       X7         X8       X9       X10      X11
                     X12         X13       X14         X15       X16
                      X17       X18
count  10000   10000      10000  10000     10000  10000      10000
    10000   10000      10000   10000     10000   10000     10000
         10000     10000       10000     10000      10000     10000
mean  0.3933  0.1901 -0.002372 0.4975   -0.018 0.4027 -0.002385
    0.2651 0.08127 -0.005998 0.01551  0.00653  0.2263 0.008542
    -0.0009043   0.0142   0.009125 0.008127 -0.004595 0.004353
std   0.4885   3.268    0.9986    0.5    1.006 0.4905    0.5763
    1.047   1.656    0.9999   1.006    1.009   1.371   0.9919
         1.167    1.005     0.7628   0.4858    0.9937    1.165
min        0  -11.13    -3.845      0   -3.546      0   -0.9995
    -3.449  -7.078    -4.129  -3.427   -3.724   -4.69   -3.968
        -4.454   -3.634     -2.984   -1.749    -4.132   -5.313
25%        0   -2.12    -0.668      0  -0.7047      0    -0.495
    -0.4457  -1.052   -0.6855 -0.6684  -0.6667 -0.7036  -0.6435
       -0.7831  -0.6657    -0.5088  -0.3218   -0.6642  -0.7883
50%        0 0.06641  -0.01109      0 -0.02551      0  -0.00853
    0.2613 0.06963   0.01027   0.015 0.005563  0.2317  0.01717
    -0.01461 0.005222 -0.0006047 0.009501  0.004249 -0.01234
```


Created by data-to-paper (AI)

Created by data-to-paper (AI)

```
75%         1   2.489    0.6733      1   0.6578      1    0.4993
    0.965    1.21    0.6609  0.6767   0.6862   1.158   0.6702
    0.7926    0.699     0.5271   0.3341    0.6645   0.7696
max         1   10.82     3.848      1    4.323      1    0.9999
    4.379   5.821     3.498   3.723    3.869   5.199    3.856
         4.527    3.726        2.83    2.097     3.805    5.315

# Categorical Variables
# Not Applicable

# Missing Values
# Not Applicable

# Additional Statistical Summaries
Skewness:
T        0.4369
Y        0.1098
X1       0.0082
X2         0.01
X3      0.05574
X4       0.3968
X5     0.005226
X6      0.02845
X7      0.02948
X8     -0.01427
X9      0.01129
X10    -0.01868
X11   -0.006799
X12    -0.05074
X13    0.006813
X14    0.009111
X15     0.02234
X16    0.004128
X17    -0.02916
X18     0.04146

Kurtosis:
T        -1.809
Y       -0.2881
X1      0.01195
X2           -2
X3     -0.00889
X4       -1.843
X5       -1.194
X6       0.0246
X7     -0.04374
X8     -0.05153
X9    0.0001571
X10    -0.01696
```

Created by data-to-paper (AI)

```
X11      -0.1151
X12       0.1375
X13     -0.01694
X14     -0.06867
X15     -0.02427
X16      0.02671
X17      0.02873
X18    -0.004264
```

C Data Analysis

C.1 Code

The Data Analysis was carried out using the following custom code:

```
# IMPORT
from my_utils import df_to_latex, df_to_figure
import pandas as pd
import numpy as np
import statsmodels.formula.api as smf
import pickle

# LOAD DATA
data = pd.read_csv('data.csv')

# DATASET PREPARATIONS
# No dataset preparations are needed.

# DESCRIPTIVE STATISTICS
# No descriptive statistics table is needed because the analysis
    ↪ focuses on specific variables X1, X4, and Y interaction.

# PREPROCESSING
# No preprocessing is needed, because the dataset has no
    ↪ categorical variables or preprocessing requirements.

# ANALYSIS

## Table df_interaction_effects:
caption = "Regression analysis for interaction effects between X1
    ↪ and X4 on Y"
# Perform regression analysis with interaction term
formula = 'Y ~ X1 * X4 + X2 + X3 + X5 + X6 + X7 + X8 + X9 + X10 +
    ↪ X11 + X12 + X13 + X14 + X15 + X16 + X17 + X18'
model = smf.ols(formula=formula, data=data).fit()

# Create a DataFrame for the regression coefficients and their
    ↪ significance
```

Created by data-to-paper (AI)

```
coef_df = pd.DataFrame({
    'coef': model.params,
    'pval': model.pvalues,
    'conf_int_low': model.conf_int()[0],
    'conf_int_high': model.conf_int()[1]
})
df_interaction_effects = coef_df.loc[['X1', 'X4', 'X1:X4']].round
    ↪ (4)
df_to_latex(df_interaction_effects, 'df_interaction_effects',
    ↪ caption=caption)

## Figure df_interaction_plot:
caption = "Interaction effect of X1 and X4 on predicted Y"
# Use specified X1 and X4 for clearer visualization of interaction
x1_levels = [-1, 0, 1]
x4_levels = [0, 1]

# Create a meshgrid from the specified levels
mesh_data = pd.DataFrame([(x, y) for x in x1_levels for y in
    ↪ x4_levels], columns=['X1', 'X4'])

# Add remaining predictors with their mean values for prediction
predictors = ['X2', 'X3', 'X5', 'X6', 'X7', 'X8', 'X9', 'X10', '
    ↪ X11', 'X12', 'X13', 'X14', 'X15', 'X16', 'X17', 'X18']
for predictor in predictors:
    mesh_data[predictor] = data[predictor].mean()

# Add interaction term
mesh_data['X1:X4'] = mesh_data['X1'] * mesh_data['X4']
mesh_data['predicted_Y'] = model.predict(mesh_data)

# Prepare DataFrame for plotting
df_interaction_plot = mesh_data.pivot_table(index='X1', columns='
    ↪ X4', values='predicted_Y')
df_interaction_plot['ci_low'] = df_interaction_plot.min(axis=1)
df_interaction_plot['ci_high'] = df_interaction_plot.max(axis=1)

df_to_figure(df_interaction_plot, 'df_interaction_plot', caption=
    ↪ caption, kind='bar', y=list(df_interaction_plot.columns)
    ↪ [:-2], y_ci=[('ci_low', 'ci_high')]*2)

# SAVE ADDITIONAL RESULTS
additional_results = {
    'Total number of observations': data.shape[0],
    'R-squared of interaction model': model.rsquared
}

with open('additional_results.pkl', 'wb') as f:
    pickle.dump(additional_results, f)
```

Created by data-to-paper (AI)

C.2 Provided Code

The code above is using the following provided functions:

```
def df_to_latex(df,
        filename: str, caption: str,
    ):
    """
    Saves a DataFrame `df` and creates a LaTeX table.
    `filename`, `caption`: as in `df.to_latex`.
    """


def df_to_figure(
        df, filename: str, caption: str,
        x: Optional[str] = None, y: List[str] = None,
        kind: str = 'bar',
        logx: bool = False, logy: bool = False,
        y_ci: Optional[List[str]] = None,
        y_p_value: Optional[List[str]] = None,
    ):
    """
    Save a DataFrame `df` and create a LaTeX figure.
    Parameters, for LaTex embedding of the figure:
    `filename`: Filename for the figure.
    `caption`: Caption for the figure.

    Parameters for df.plot():
    `x`: Column name for x-axis (index by default).
    `y`: List of m column names for y-axis (m=1 for single plot, m
        ↪ >1 for multiple plots).
    `kind`: only bar is allowed.
    `logx` / `logy` (bool): log scale for x/y axis.

    `y_ci`: Confidence intervals for errorbars.
        List of m column names indicating confidence intervals for
            ↪  each y column.
        Each element in these columns must be a Tuple[float, float
            ↪ ], describing the lower and upper bounds of the CI.

     `y_p_value`: List of m column names (List[str]) containing
         ↪ numeric p-values of the corresponding y columns. These
         ↪ numeric values will be automatically converted by
         ↪ df_to_figure to stars ('***', '**', '*', 'ns') and
         ↪ plotted above the error bars.

    If provided, the length of `y_ci`, and `y_p_value` should be
        ↪ the same as of `y`.
```

Created by data-to-paper (AI)

```
Example:
Suppose, we have:

df_lin_reg_longevity = pd.DataFrame({
    'adjusted_coef': [0.4, ...], 'adjusted_coef_ci': [(0.35,
        ↪ 0.47), ...], 'adjusted_coef_pval': [0.012, ...],
    'unadjusted_coef': [0.2, ...], 'unadjusted_coef_ci':
        ↪ [(0.16, 0.23), ...], 'unadjusted_coef_pval':
        ↪ [0.0001, ...],
}, index=['var1', ...])

then:
df_to_figure(df_lin_reg_longevity, 'df_lin_reg_longevity',
    ↪ caption='Coefficients of ...', kind='bar',
    y=['adjusted_coef', 'unadjusted_coef'],
    y_ci=['adjusted_coef_ci', 'unadjusted_coef_ci'],
    y_p_value=['adjusted_coef_pval', 'unadjusted_coef_pval'])
"""
```

C.3 Code Description

This code performs an analysis integrating regression modeling to explore the interaction effects of two selected variables, specifically $X1$ and $X4$, on the response variable Y. The broader goal is to discern the impact of inter-variable interplay in conjunction with other predictors within the dataset.

C.4 Regression Analysis

The primary focus of the analysis is to gauge the interaction effects between the predictor variables $X1$ and $X4$ on the response variable Y. This was achieved through a regression model that includes the main effects of $X1$ and $X4$ as well as their interaction term. The model formula is defined as $Y \sim X1 \times X4 + X2 + X3 + \ldots + X18$, where $\times$ denotes the interaction between $X1$ and $X4$. This regression model was constructed using the ordinary least squares (OLS) method from the `statsmodels` library. The outcome of this regression provides coefficients, significance levels, and confidence intervals for each term within the model. The model's key interest is the interaction term $X1 : X4$, and a concise table of results focusing on this interaction is rendered for further interpretation.

C.5 Interaction Effect Visualization

The code includes a visualization step to provide insights into how the interaction between $X1$ and $X4$ influences the predicted values of Y. A scenario-

Created by data-to-paper (AI)

based approach was adopted, focusing on specific levels for $X1$ and $X4$: $X1$ levels are set to -1, 0, and 1, while $X4$ levels are binary, i.e., 0 and 1. A grid was constructed using these levels to simulate various combinations of $X1$ and $X4$. For the purpose of model prediction, other predictors ($X2$, $X3$, etc.) were set to their mean values. The predictions from the fitted model were then utilized to construct a pivot table. This table captures the predicted values and their potential confidence intervals, subsequently visualized using a bar plot to effectively communicate the interaction effects on Y.

C.6 Summary and Outputs

Concluding the analysis, the code computes and stores supplementary metrics including the total number of observations in the dataset and the R-squared value of the interaction model. These metrics are serialized and saved for documentation or subsequent analysis. Overall, the methodology employed seeks to elucidate the dynamics and significance of interaction effects in multivariable regression contexts, ultimately enriching the interpretative potential concerning the datasets interrelated variables.

C.7 Code Output

df_interaction_effects.pkl

```
         coef       pval conf_int_low conf_int_high
X1      1.141          0         1.11         1.172
X4    -0.8683  2.39e-277      -0.9147        -0.822
X1:X4  0.0246       0.29       -0.021        0.0702
```

df_interaction_plot.pkl

```
X4       0       1  ci_low ci_high
X1
-1 -0.5986  -1.492  -1.492 -0.5986
 0  0.5424 -0.3259 -0.3259  0.5424
 1   1.683  0.8397  0.8397   1.683
```

additional_results.pkl

```
{
    'Total number of observations': 10000,
    'R-squared of interaction model': 0.8792,
}
```

Created by data-to-paper (AI)

D LaTeX Table Design

D.1 Code

The LaTeX Table Design was carried out using the following custom code:

```
# IMPORT
import pandas as pd
from my_utils import df_to_latex, df_to_figure, is_str_in_df,
    ↪ split_mapping, AbbrToNameDef

# PREPARATION FOR ALL TABLES AND FIGURES
# Define mapping for all df labels that need to be renamed and/or
    ↪ glossary defined.
all_mapping: AbbrToNameDef = {
    # Variables and their definitions
    'X1': ('Variable X1', 'A numerical predictor variable.'),
    'X4': ('Variable X4', 'Another numerical predictor variable.')
        ↪ ,
    'Y': ('Outcome Variable Y', 'The dependent variable we are
        ↪ interested in predicting.'),
    'coef': ('Coefficient', 'Estimated regression coefficient'),
    'pval': ('P-value', 'Statistical significance of the
        ↪ coefficient'),
    'conf_int_low': ('Lower CI', 'Lower bound of 95% confidence
        ↪ interval for coefficient'),
    'conf_int_high': ('Upper CI', 'Upper bound of 95% confidence
        ↪ interval for coefficient'),
    0: ('X4=0', 'Predicted Y when X4 is 0'),
    1: ('X4=1', 'Predicted Y when X4 is 1'),
    'ci_low': ('Lower Predicted CI', 'Lower bound of predicted Y
        ↪ confidence interval'),
    'ci_high': ('Upper Predicted CI', 'Upper bound of predicted Y
        ↪ confidence interval'),
    'X1:X4': ('Interaction X1:X4', 'Interaction term between X1
        ↪ and X4'),
}

# Process df_interaction_effects:
df_interaction_effects = pd.read_pickle('df_interaction_effects.
    ↪ pkl')

# Format values:
# Not Applicable

# Rename rows and columns:
# Get from the `all_mapping` the labels that are in the focal df:
```

Created by data-to-paper (AI)

```python
mapping = dict((k, v) for k, v in all_mapping.items() if
    ↪ is_str_in_df(df_interaction_effects, k))
abbrs_to_names, glossary = split_mapping(mapping)
df_interaction_effects.rename(columns=abbrs_to_names, index=
    ↪ abbrs_to_names, inplace=True)

# As a table:
df_to_latex(
    df_interaction_effects, 'df_interaction_effects_formatted',
    caption="Regression Interaction Effects Between Variable X1
        ↪ and X4 on Outcome Y",
    note=None,
    glossary=glossary)

# Process df_interaction_plot:
df_interaction_plot = pd.read_pickle('df_interaction_plot.pkl')

# Format values:
# Not Applicable

# Rename rows and columns:
# Get from the `all_mapping` the labels that are in the focal df:
mapping = dict((k, v) for k, v in all_mapping.items() if
    ↪ is_str_in_df(df_interaction_plot, k))
abbrs_to_names, glossary = split_mapping(mapping)
df_interaction_plot.rename(columns=abbrs_to_names, index=
    ↪ abbrs_to_names, inplace=True)

# As a figure:
df_to_figure(
    df_interaction_plot, 'df_interaction_plot_formatted',
    caption="Interaction Effect of X1 and X4 on Predicted Outcome
        ↪ Y",
    note="This figure illustrates the predicted outcome Y under
        ↪ different levels of X1 and X4, showing interaction
        ↪ effects.",
    glossary={k: glossary[k] for k in ['X4=0', 'X4=1', 'Lower
        ↪ Predicted CI', 'Upper Predicted CI']},
    kind='bar',
    y=['X4=0', 'X4=1'],
    y_ci=[('Lower Predicted CI', 'Upper Predicted CI')]*2,
    xlabel='Levels of X1',
    ylabel='Predicted Y'
)
```

D.2 Provided Code

The code above is using the following provided functions:

Created by data-to-paper (AI)

```python
def df_to_latex(df,
        filename: str, caption: str,
        note: str = None,
        glossary: Dict[Any, str] = None,
    ):
    """
    Saves a DataFrame `df` and creates a LaTeX table.
    `filename`, `caption`: as in `df.to_latex`.
    `note`: Note to be added below the table caption.
    `glossary`: Glossary for the table.
    """


def df_to_figure(
        df, filename: str, caption: str,
        note: str = None, glossary: Dict[Any, str] = None,
        x: Optional[str] = None, y: List[str] = None,
        kind: str = 'bar',
        logx: bool = False, logy: bool = False,
        y_ci: Optional[List[str]] = None,
        y_p_value: Optional[List[str]] = None,
        xlabel: str = None, ylabel: str = None,
    ):
    """
    Save a DataFrame `df` and create a LaTeX figure.
    Parameters, for LaTex embedding of the figure:
    `filename`: Filename for the figure.
    `caption`: Caption for the figure.
    `note`: Note to be added below the figure caption.
    `glossary`: Glossary for the figure.

    Parameters for df.plot():
    `x`: Column name for x-axis (index by default).
    `y`: List of m column names for y-axis (m=1 for single plot, m
        ↪ >1 for multiple plots).
    `kind`: only bar is allowed.
    `logx` / `logy` (bool): log scale for x/y axis.
    `xlabel`: Label for the x-axis.
    `ylabel`: Label for the y-axis.

    `y_ci`: Confidence intervals for errorbars.
        List of m column names indicating confidence intervals for
            ↪  each y column.
        Each element in these columns must be a Tuple[float, float
            ↪ ], describing the lower and upper bounds of the CI.

     `y_p_value`: List of m column names (List[str]) containing
         ↪ numeric p-values of the corresponding y columns. These
         ↪ numeric values will be automatically converted by
```

Created by data-to-paper (AI)

```
        ↪ df_to_figure to stars ('***', '**', '*', 'ns') and
        ↪ plotted above the error bars.

    If provided, the length of `y_ci`, and `y_p_value` should be
        ↪ the same as of `y`.

    Example:
    Suppose, we have:

    df_lin_reg_longevity = pd.DataFrame({
        'adjusted_coef': [0.4, ...], 'adjusted_coef_ci': [(0.35,
            ↪ 0.47), ...], 'adjusted_coef_pval': [0.012, ...],
        'unadjusted_coef': [0.2, ...], 'unadjusted_coef_ci':
            ↪ [(0.16, 0.23), ...], 'unadjusted_coef_pval':
            ↪ [0.0001, ...],
    }, index=['var1', ...])

    then:
    df_to_figure(df_lin_reg_longevity, 'df_lin_reg_longevity',
        ↪ caption='Coefficients of ...', kind='bar',
        y=['adjusted_coef', 'unadjusted_coef'],
        y_ci=['adjusted_coef_ci', 'unadjusted_coef_ci'],
        y_p_value=['adjusted_coef_pval', 'unadjusted_coef_pval'])
    """


def is_str_in_df(df: pd.DataFrame, s: str):
    return any(s in level for level in getattr(df.index, 'levels',
        ↪  [df.index]) + getattr(df.columns, 'levels', [df.columns
        ↪ ]))

AbbrToNameDef = Dict[Any, Tuple[Optional[str], Optional[str]]]

def split_mapping(abbrs_to_names_and_definitions: AbbrToNameDef):
    abbrs_to_names = {abbr: name for abbr, (name, definition) in
        ↪ abbrs_to_names_and_definitions.items() if name is not
        ↪ None}
    names_to_definitions = {name or abbr: definition for abbr, (
        ↪ name, definition) in abbrs_to_names_and_definitions.
        ↪ items() if definition is not None}
    return abbrs_to_names, names_to_definitions
```

D.3 Code Output

df_interaction_effects_formatted.pkl

```
\begin{table}[h]
```

Created by data-to-paper (AI)

```
\caption{Regression Interaction Effects Between Variable X1 and X4
     on Outcome Y}
\label{table:df-interaction-effects-formatted}
\begin{threeparttable}
\renewcommand{\TPTminimum}{\linewidth}
\makebox[\linewidth]{%
\begin{tabular}{lrlrr}
\toprule
 & Coefficient & P-value & Lower CI & Upper CI \\
\midrule
\textbf{Variable X1} & 1.141 & $<$1e-06 & 1.11 & 1.172 \\
\textbf{Variable X4} & -0.8683 & $<$1e-06 & -0.9147 & -0.822 \\
\textbf{Interaction X1:X4} & 0.0246 & 0.29 & -0.021 & 0.0702 \\
\bottomrule
\end{tabular}}
\begin{tablenotes}
\footnotesize
\item \textbf{Variable X1}: A numerical predictor variable.
\item \textbf{Variable X4}: Another numerical predictor variable.
\item \textbf{Coefficient}: Estimated regression coefficient
\item \textbf{P-value}: Statistical significance of the
    coefficient
\item \textbf{Lower CI}: Lower bound of 95\% confidence interval
    for coefficient
\item \textbf{Upper CI}: Upper bound of 95\% confidence interval
    for coefficient
\item \textbf{Interaction X1:X4}: Interaction term between X1 and
    X4
\end{tablenotes}
\end{threeparttable}
\end{table}
```

df_interaction_plot_formatted.pkl

```
\begin{figure}[htbp]
\centering
\includegraphics{df_interaction_plot_formatted.png}
\caption{Interaction Effect of X1 and X4 on Predicted Outcome Y
This figure illustrates the predicted outcome Y under different
    levels of X1 and X4, showing interaction effects.
X4=0: Predicted Y when X4 is 0.
X4=1: Predicted Y when X4 is 1.
Lower Predicted CI: Lower bound of predicted Y confidence interval
    .
Upper Predicted CI: Upper bound of predicted Y confidence interval
    .}
\label{figure:df-interaction-plot-formatted}
\end{figure}
% This latex figure presents "df_interaction_plot_formatted.png",
```

Created by data-to-paper (AI)

```
% which was created from the df:
%
% index,"X4=0","X4=1","Lower Predicted CI","Upper Predicted CI"
% -1,-0.5986,-1.492,-1.492,-0.5986
% "X4=0",0.5424,-0.3259,-0.3259,0.5424
% "X4=1",1.683,0.8397,0.8397,1.683
%
% To create the figure, this df was plotted with the command:
%
% df.plot(kind='bar', y=['X4=0', 'X4=1'], xlabel='Levels of X1',
    ylabel='Predicted Y')
%
% Confidence intervals for y-values were then plotted based on
    column: [('Lower Predicted CI', 'Upper Predicted CI'), ('Lower
     Predicted CI', 'Upper Predicted CI')].
```